TE3 – UML Class Diagram

LedgerLogic (Finance Budget Tracker) · CSC 3100 · Fall 2026 · Team OpenAnthropic

Team	Shreev B. (Lead Developer) · Juan S. (Scrum Master & PM) · Juan F. (Project Tester)
Repository	https://github.com/shreev2703/finance-budget-tracker
Date	October 16, 2026

LedgerLogic is a web app that helps college students log income and expenses, see their monthly balance, and stay under per-category spending limits, with a warning at 80% of a limit. This document contains all of our TE3 diagrams: the UML class diagram for our domain model, how each user story maps to the classes, our initial UML sketch, and our UI mockups.

1. UML Class Diagram (Domain Model)

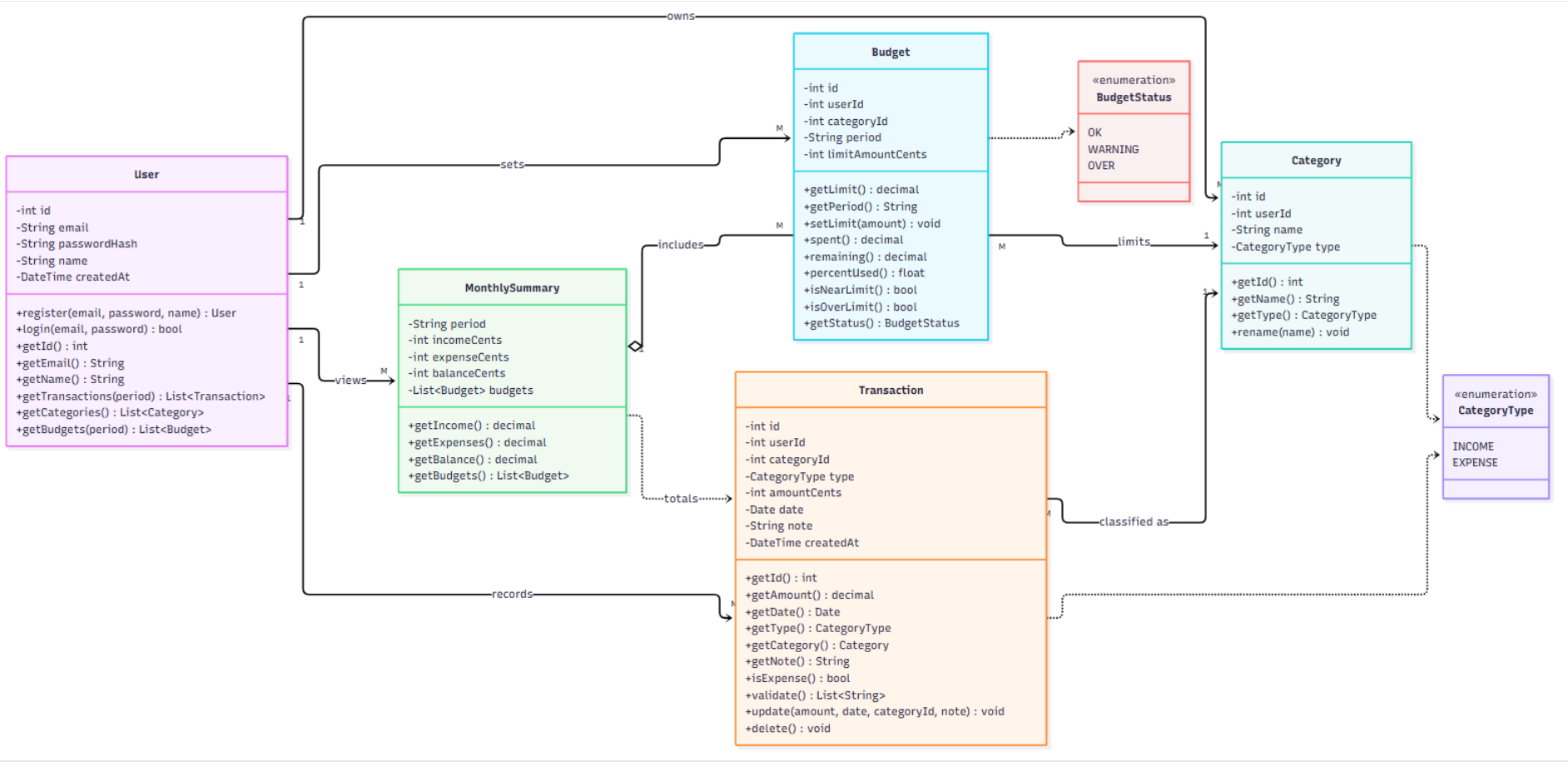


Figure A. LedgerLogic UML class diagram (domain model). Source: `docs/diagram-sources/te3/class-diagram.mmd`.

How to read the diagram

Notation	Meaning
- (minus)	**Private** attribute. Every attribute is private, so other classes cannot read or change an object's data directly.
+ (plus)	**Public** method: the only way other classes use an object (getters such as `getAmount()`, and operations such as `setLimit()` and `validate()`).
`1` / `M`	Multiplicity: `1` = exactly one, `M` = many (zero or more). Example: one User records M Transactions; each Transaction belongs to 1 Category.
Solid arrow	Association: the class at the tail holds a reference to the class at the head.
Hollow diamond	Aggregation: a MonthlySummary includes the Budgets for that month.
Dashed arrow	Dependency: the class uses the enumeration or reads the other class's data.

Classes

Class	Responsibility and key data	In the app today	Stories
User	A registered student who owns all of their data. Stores email, a password hash (never the password), name, and when the account was created. `register()` and `login()` handle accounts; the getters return only that user's transactions, categories, and budgets.	`users` table; the demo uses a seeded user until login is built	US-01, US-02, US-07
Category	A user-defined bucket such as "Groceries" or "Part-time Job". Belongs to one user (`userId`) and is typed INCOME or EXPENSE.	`categories` table; `GET /api/categories`; Create / Edit Category screen	US-01, US-02, US-05, US-06, US-08
Transaction	One income or expense: the **amount** (stored in cents), the **date** it happened, a note, its type and category, the owner, and when it was entered (`createdAt`). `validate()` checks amount > 0, a valid date, and that the category matches the type.	`transactions` table; `GET / POST /api/transactions`; Transactions screen	US-01 – US-06
Budget	A monthly spending limit (in cents) for one expense category and one month (`period` = YYYY-MM). Calculates spent, remaining, and percent used, and returns a `BudgetStatus`.	`budgets` table; `domain/budget.js`; dashboard category bars	US-08, US-09
MonthlySummary	The dashboard view for one month: total income, total expenses, remaining balance, and the status of every budget. Computed from Transactions and Budgets, not stored.	`GET /api/budgets/summary`; dashboard budget card	US-04, US-08, US-09
CategoryType	Enumeration: `INCOME` or `EXPENSE`.	CHECK constraint on `type`	US-01, US-02
BudgetStatus	Enumeration: `OK`, `WARNING` (80% or more used), `OVER` (above 100%).	`budgetStatus()` in `domain/budget.js`	US-09

Design notes. Money is stored as integer cents (`amountCents`, `limitAmountCents`) to avoid rounding errors; the public getters return dollars. Every class except the enumerations carries a `userId`, so each user only ever sees their own data. A user has at most one budget per category per month. Filtering history (US-06) is a query through `User.getTransactions(period)`, not a separate class. A SavingsGoal class was considered and left out because no TE2 story requires it.

2. Story → Class Traceability

Story	Summary	Classes	Attributes / methods
US-01	Log an expense with amount, date, category, description	Transaction, Category, User	`Transaction.amountCents`, `date`, `note`, `type = EXPENSE`; `validate()`; Transaction → Category
US-02	Record paychecks and income sources	Transaction, Category, User	`Transaction.type = INCOME`; income categories (e.g. "Part-time Job")
US-03	Edit or delete a past transaction	Transaction	`update(amount, date, categoryId, note)`, `delete()`
US-04	Dashboard: income, expenses, remaining balance	MonthlySummary, Transaction	`getIncome()`, `getExpenses()`, `getBalance()`
US-05	Spending-by-category chart	Transaction, Category	Expense `amountCents` grouped by Category for the month
US-06	Filter history by category and date range	User, Transaction, Category	`User.getTransactions(period)` filtered by `categoryId` and `date`
US-07	Create an account and log in	User	`email`, `passwordHash`, `register()`, `login()`
US-08	Monthly spending limit per category	Budget, Category	`limitAmountCents`, `period`; `setLimit()`, `remaining()`, `isOverLimit()`
US-09	Warning at 80% of a category limit	Budget, BudgetStatus	`percentUsed()`, `isNearLimit()`, `getStatus()` → `WARNING`

3. Initial UML Sketch

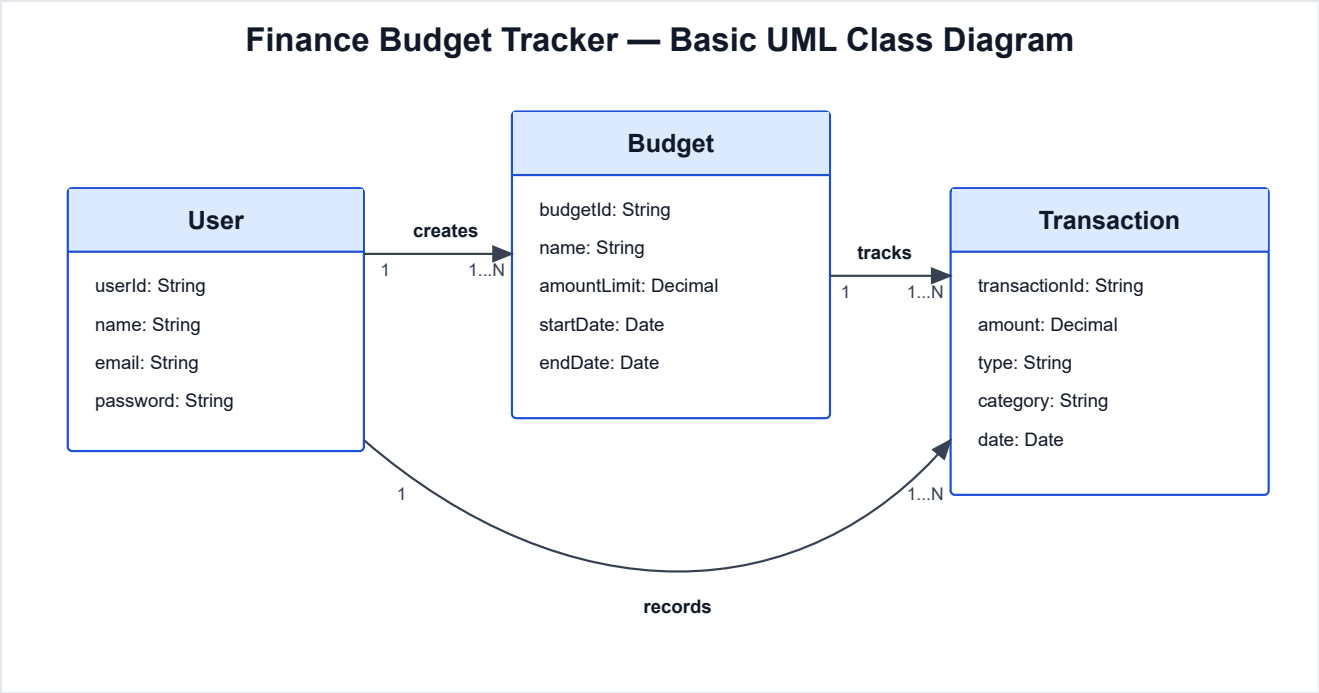


Figure B. Initial UML sketch by Juan S. (User, Budget, Transaction). Figure A refines it: categories became their own class, budgets became per category per month, attributes became private with public methods, and the INCOME / EXPENSE type and budget status became enumerations.

4. UI Mockups (Wireframes)

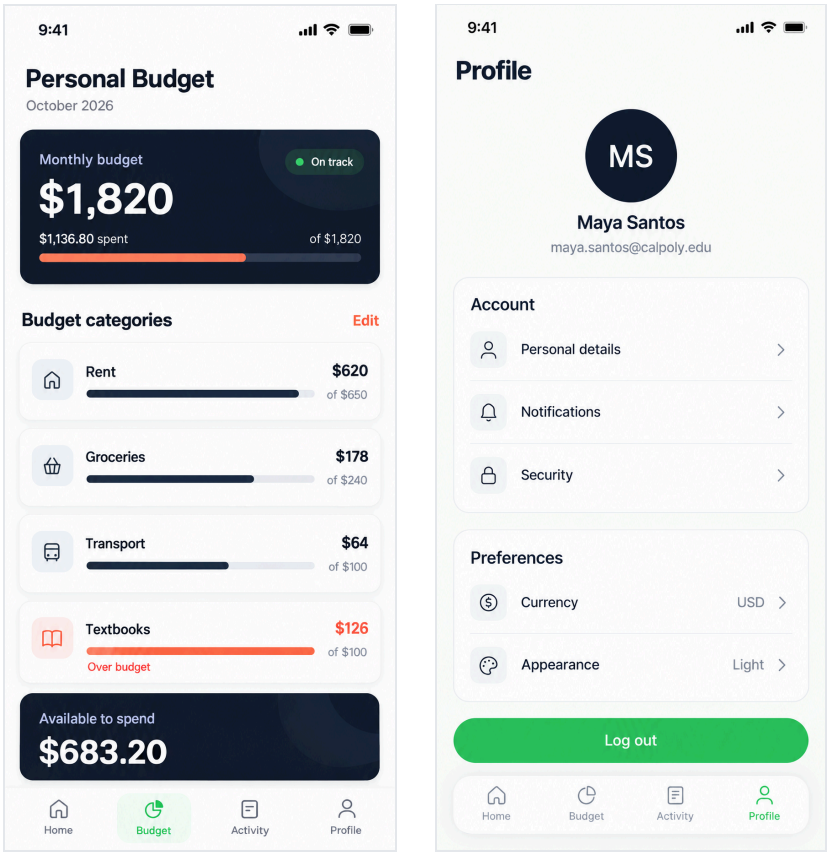


Figure C. UI mockups by Juan S.: Budget screen (monthly budget, category bars, over-budget state) and Profile screen.